

INSTRUÇÕES DE CORREÇÃO — DOIS BLOQUEADORES DE UX

Relatório Técnico de Correção e Diretrizes de Implementação Permanente

08 de Agosto de 2026

INSTRUÇÕES DE CORREÇÃO — DOIS BLOQUEADORES DE UX

Data: 08 de Agosto de 2026

Projeto: retrabalho-system-permanente

Arquivo afetado: client/src/pages/logistica/ (Solicitacoes.tsx e/ou NovaCotacaoDialog.tsx)

Prioridade: ALTA — Experiência do usuário está comprometida

1. ESCOPO DO DOCUMENTO E DIRETRIZES TÉCNICAS

Este documento estabelece as diretrizes técnicas obrigatórias para a correção de dois bloqueadores críticos de experiência do usuário (UX) identificados no módulo de logística. A natureza destas correções exige uma abordagem de engenharia de software que priorize a persistência de dados e a eficiência de consultas em banco de dados, visando a estabilidade a longo prazo do sistema **retrabalho-system-permanente**.

A fundamentação legal para a manutenção da integridade e disponibilidade dos dados baseia-se no Art. 7º, inciso II da Lei nº 13.709/2018 (LGPD), que pressupõe a execução de contratos e procedimentos preliminares com eficiência tecnológica. A falha na manutenção de um sistema funcional e performático constitui negligência técnica, podendo resultar em perdas operacionais significativas e violação de SLAs internos estabelecidos entre os departamentos de TI e Logística.

As consequências do descumprimento destas instruções incluem a persistência da latência de rede, o aumento do retrabalho por parte dos operadores de logística e a degradação da confiança no sistema **Manus Iago**. Portanto, cada seção a seguir deve ser implementada com rigor técnico, garantindo que as funcionalidades não sofram retrocessos em atualizações futuras.

2. BLOQUEADOR 1: LATÊNCIA EXCESSIVA NO CARREGAMENTO DE SOLICITAÇÕES

O sintoma relatado consiste em um tempo de resposta proibitivo ao acessar o menu "Solicitação de Frete". A análise técnica identificou que o sistema realiza uma varredura completa (full table scan) na tabela de solicitações, ignorando parâmetros de temporalidade. Esta prática viola princípios básicos de escalabilidade e performance de sistemas distribuídos, sobrecarregando tanto o servidor de aplicação quanto o banco de dados.

A base legal para esta correção encontra-se no princípio da eficiência administrativa e operacional. O *Código Civil*, em seu Art. 389, estabelece a responsabilidade pelo cumprimento das obrigações, o que no contexto de software traduz-se na entrega de uma ferramenta que cumpra sua função em tempo hábil. A ausência de filtros de data resulta em um processamento desnecessário de dados históricos que não possuem relevância imediata para a operação cotidiana.

A inobservância desta correção acarretará o travamento completo do módulo de logística à medida que o volume de dados cresce. O desenvolvedor deve garantir que a query SQL gerada pelo ORM Prisma contenha as cláusulas **WHERE** específicas para o intervalo de 30 dias, limitando o tráfego de rede e o consumo de memória no frontend.

3. ANÁLISE DO CÓDIGO DEFEITUOSO E IDENTIFICAÇÃO DE FALHAS

O código atual apresenta uma falha estrutural onde as variáveis de controle de tempo são declaradas, porém não são injetadas na lógica de consulta ao banco de dados. Esta desconexão entre a interface e o backend demonstra uma falha na cadeia de responsabilidade do processamento de dados, onde o sistema "puxa" informações desnecessárias por padrão.

O erro técnico reside na omissão de operadores lógicos de comparação. Conforme as melhores práticas de desenvolvimento *Clean Code* e arquitetura de microsserviços, toda listagem deve ser paginada ou filtrada por padrão. A manutenção de uma consulta sem limites (unbounded query) é considerada uma vulnerabilidade de performance que pode ser explorada para causar negação de serviço (DoS) interna.

A correção exige a reestruturação da chamada ao Prisma para incluir os operadores **gte** (greater than or equal) e **lte** (less than or equal). O não cumprimento desta diretriz manterá o sistema em estado de degradação contínua, impedindo a expansão da base de dados sem comprometer a usabilidade da interface.

4. IMPLEMENTAÇÃO DO FILTRO TEMPORAL E LIMITAÇÃO DE REGISTROS

A correção necessária envolve a definição de um intervalo dinâmico de 30 dias retroativos a partir da data atual. Este procedimento deve ser executado no lado do servidor (Server-side) para garantir que apenas o subconjunto necessário de dados seja transmitido pela rede. A implementação deve seguir o padrão de segurança de dados, garantindo que o estado da aplicação reflita a realidade operacional mais recente.

```
// CORRIGIDO – busca apenas últimos 30 dias por padrão
const agora = new Date();
const trintaDiasAtras = new Date(agora);
trintaDiasAtras.setDate(trintaDiasAtras.getDate() - 30);

const solicitacoes = await prisma.solicitacaoFrete.findMany({
  where: {<br/>
    createdAt: {<br/>
      gte: trintaDiasAtras, // maior ou igual a 30 dias atrás<br/>
      lte: agora,          // menor ou igual a agora
    },
  },
  orderBy: { createdAt: 'desc' },<br/>
  take: 100, // limite de segurança: máximo 100 registros
});
```

A inclusão da cláusula **take: 100** serve como uma salvaguarda técnica contra picos de inserção de dados. Esta medida previne que, mesmo dentro do período de 30 dias, um volume anômalo de registros degrade a performance do navegador do usuário final. O descumprimento desta trava de segurança será considerado uma falha na implementação da arquitetura de defesa em profundidade.

Após a implementação, o comportamento esperado é uma redução drástica no *Time to First Byte (TTFB)* e no tempo de renderização total da página. O sistema deve ser capaz de processar a solicitação e exibir os dados em menos de 2 segundos, atendendo aos requisitos de performance modernos para aplicações corporativas.

5. BLOQUEADOR 2: VOLATILIDADE DE ESTADO E PERDA DE DADOS EM BACKGROUND

O segundo bloqueador refere-se à perda de dados do formulário quando o usuário alterna entre abas do navegador Chrome. Este fenômeno ocorre devido à política de gerenciamento de memória do navegador e ao ciclo de vida de componentes React, que desmontam elementos inativos para preservar recursos do sistema operacional. A ausência de uma camada de persistência local torna o trabalho do usuário volátil e inseguro.

Juridicamente, a perda de dados inseridos pelo usuário sem aviso prévio ou mecanismo de recuperação pode ser interpretada como uma falha na prestação do serviço tecnológico, conforme o *Código de Defesa do Consumidor (aplicado subsidiariamente em relações B2B)*. A expectativa de confiabilidade é um pilar do software profissional, e a volatilidade do estado do formulário rompe este princípio fundamental.

A consequência direta desta falha é o aumento do custo operacional, visto que o tempo gasto pelo colaborador para preencher novamente os dados é um desperdício de recursos da empresa. A solução exige a implementação de um mecanismo de

"Hydration" de estado, onde os dados são espelhados em um armazenamento persistente de curta duração (`sessionStorage`).

6. MECANISMO DE PERSISTÊNCIA VIA SESSIONSTORAGE

A utilização do **`sessionStorage`** é a escolha técnica correta para este cenário, pois mantém os dados vivos enquanto a sessão do navegador (aba) estiver aberta, mas os limpa automaticamente quando a aba é fechada, garantindo a segurança e a privacidade dos dados logísticos. Diferente do `localStorage`, o `sessionStorage` evita a poluição de dados entre diferentes abas ou sessões de usuários distintos no mesmo terminal.

O desenvolvedor deve implementar um *hook* de sincronização que monitore as mudanças no estado do formulário e as serialize para o formato JSON em tempo real. Este procedimento garante que, em caso de desmontagem inesperada do componente por falta de memória ou troca de contexto, o estado possa ser restaurado integralmente no momento da remontagem (`useEffect` de inicialização).

A falha em implementar este mecanismo de persistência resultará na manutenção da insatisfação do usuário e na ineficiência do processo de cotação de frete. É imperativo que a restauração de dados seja silenciosa e automática, proporcionando uma experiência de uso contínua e sem fricções, independentemente das ações de gerenciamento de recursos do navegador.

7. PERSISTÊNCIA VISUAL E CONTROLE DE MONTAGEM DO DIALOG

Além da persistência de dados, é necessário alterar a forma como o componente de interface (Dialog/Modal) interage com a árvore do DOM (Document Object Model). A prática de desmontar completamente o componente ao fechar ou ao perder o foco é eficiente para memória, mas prejudicial para a experiência do usuário em formulários complexos. A implementação deve utilizar propriedades de montagem forçada ou controle de visibilidade via CSS.

Ao manter o Dialog montado, preservamos o estado interno do React sem depender exclusivamente do armazenamento externo para elementos de interface menores (como foco de campos ou posição de scroll). Esta técnica, combinada com a persistência em `sessionStorage`, cria uma redundância de segurança que blinda a aplicação contra falhas de renderização.

O uso de propriedades como **`onInteractOutside`** e **`onEscapeKeyDown`** deve ser configurado para prevenir o fechamento acidental. O sistema deve solicitar confirmação explícita do usuário caso existam dados preenchidos e ele tente fechar o formulário, aplicando o princípio da prevenção de erros da heurística de Nielsen.

8. PROTOCOLO DE LIMPEZA E FINALIZAÇÃO DE CICLO DE DADOS

Um sistema de persistência só é completo se possuir um protocolo rigoroso de limpeza de dados (garbage collection manual). Após a submissão bem-sucedida do formulário de solicitação de frete, o sistema deve obrigatoriamente limpar o **sessionStorage**. A permanência de dados de uma solicitação anterior em um novo formulário é um erro grave que pode levar ao envio de informações incorretas para transportadoras.

Esta obrigação de limpeza fundamenta-se na precisão dos dados, conforme exigido pela *LGPD*. Dados residuais podem causar confusão operacional e erros de faturamento. Portanto, a função de sucesso da submissão (`handleSubmitSuccess`) deve ser o único ponto de entrada para a destruição do estado persistido, garantindo que o ciclo de vida do dado seja respeitado do início ao fim.

A implementação deve incluir logs de depuração para confirmar que a limpeza ocorreu. Caso o usuário cancele a operação voluntariamente, o sistema também deve oferecer a opção de limpar os dados ou mantê-los para uma conclusão posterior, dando total controle da informação ao operador humano.

9. MATRIZ DE TESTES EXAUSTIVOS E VALIDAÇÃO TÉCNICA

A validação das correções deve seguir um protocolo de testes de estresse e de usabilidade. Para o Bloqueador 1, deve-se utilizar ferramentas de monitoramento de rede (Network tab do Chrome) para verificar o tamanho do payload retornado e o tempo de execução da query no banco de dados. Qualquer resposta acima de 2 segundos será considerada falha no teste de aceitação.

Para o Bloqueador 2, o teste deve simular condições extremas de uso: preenchimento parcial, troca de abas, abertura de múltiplas abas simultâneas e simulação de queda de memória do navegador. A integridade dos dados deve ser de 100% em todos os cenários. A perda de um único campo (ex: peso ou cubagem) invalidará a correção.

Os testes de regressão são obrigatórios. O desenvolvedor deve garantir que a implementação do filtro de data não quebrou a funcionalidade de busca por outros critérios e que a persistência no `sessionStorage` não interfere em outros formulários do sistema. A documentação dos resultados dos testes deve ser anexada ao log de deploy.

10. TABELA RESUMIDA DE IMPLEMENTAÇÃO

ID	Bloqueador	Ação Corretiva	Impacto Esperado
UX-01	Lentidão na Listagem	Filtro Prisma gte/lte (30 dias) + take: 100	Carregamento em < 2s e redução de carga no DB
UX-02	Perda de Dados (Abas)	Persistência em sessionStorage + Dialog persistente	Zero perda de dados ao alternar contextos

ID	Bloqueador	Ação Corretiva	Impacto Esperado
UX-03	Integridade de Dados	Limpeza obrigatória pós-sucesso	Prevenção de duplicidade ou erro de preenchimento

11. CONSIDERAÇÕES FINAIS E COMPROMISSO DE NÃO-REGRESSÃO

As correções aqui detalhadas são fundamentais para a continuidade operacional do sistema **Manus Iago**. A arquitetura proposta resolve não apenas os sintomas imediatos, mas ataca as causas raízes de performance e volatilidade de estado. É responsabilidade do desenvolvedor sênior garantir que estas práticas se tornem o novo padrão para todos os módulos de formulários do sistema.

A regressão destas funcionalidades em atualizações futuras será tratada como falha grave de controle de versão e qualidade. Recomenda-se a criação de testes automatizados (E2E com Cypress ou Playwright) que cubram especificamente o cenário de persistência de dados em diálogos e a performance de carregamento da página de logística.

Este brief técnico serve como guia definitivo para a resolução dos problemas apontados, devendo ser seguido à risca para garantir a entrega de um software robusto, confiável e alinhado com as necessidades críticas do departamento de logística da organização.

COORDENAÇÃO DE SISTEMAS

DESENVOLVIMENTO SÊNIOR

Local e data: São Paulo, 08 de Agosto de 2026

Documento elaborado em 08 de Agosto de 2026. As informações contidas são de responsabilidade do solicitante.

12. PROTOCOLO DE TRATAMENTO DE ERROS NA COMUNICAÇÃO COM A API MUBISYS

A integração com a API externa MubiSys exige uma camada de resiliência que o sistema atualmente não possui. A falha na captura de erros de autenticação ou de expiração de token resulta em falhas silenciosas no frontend, onde o usuário visualiza campos vazios sem compreender que a origem do problema é uma falha de comunicação. Conforme o *Art. 20 do Código de Defesa do Consumidor*, a falta de informação clara sobre falhas na prestação de serviços configura vício de qualidade, o que em software se traduz na necessidade de Error Boundaries e notificações de falha explícitas.

A fundamentação técnica para esta seção baseia-se na necessidade de validar o corpo da resposta HTTP, independentemente do Status Code retornado. Identificou-se que a API MubiSys pode retornar um código 200 (OK) mesmo quando ocorre um erro interno, enviando a mensagem de erro dentro do objeto JSON. O desenvolvedor deve implementar um interceptor ou uma função de validação que verifique a existência de propriedades como `error` ou `message` antes de tentar realizar o *destructuring* dos dados de CNPJ e Razão Social.

A consequência da não implementação deste protocolo é a persistência do "Problema 1" relatado pelo usuário, onde o sistema tenta ler propriedades de um objeto `undefined`, causando quebras na interface (White Screen of Death) ou preenchimento nulo de campos obrigatórios. É obrigatória a implementação de um `try-catch` robusto em torno da chamada de busca de OS, com um `toast.error` informativo que instrua o usuário a verificar a validade do token ou a conexão com o servidor MubiSys.

13. GESTÃO DE TOKENS E CREDENCIAIS EM AMBIENTE DE PRODUÇÃO

A análise do código revelou a presença de tokens JWT hardcoded, com data de expiração fixa. Esta prática é uma violação grave das normas de segurança da informação, especificamente da *ISO/IEC 27001*, que trata do controle de acesso e gestão de segredos. A expiração de um token embutido no código causa a interrupção imediata de todas as funcionalidades de logística, exigindo um novo deploy apenas para atualizar uma credencial, o que é inaceitável em um ambiente de missão crítica.

A correção exige a migração imediata de todas as chaves de API e tokens para variáveis de ambiente (`.env`), gerenciadas pelo servidor e nunca expostas diretamente no código-fonte do frontend. O racional técnico é permitir a rotação de credenciais sem a necessidade de alteração na lógica da aplicação, garantindo a continuidade do serviço. Além disso, deve-se implementar uma lógica de renovação automática ou, no mínimo, um alerta administrativo quando o token estiver a menos de 15 dias da expiração.

O descumprimento desta diretriz de segurança sujeita o sistema a paradas não programadas, como a ocorrida na data presente. A responsabilidade pela manutenção das credenciais deve ser centralizada em um serviço de configuração ou cofre de senhas (Vault), impedindo que desenvolvedores tenham acesso direto a tokens de produção e garantindo que o sistema Manus ligo permaneça operacional independentemente de datas de expiração fixas em código legado.

14. OTIMIZAÇÃO DE ESTADO GLOBAL E SINCRONIZAÇÃO DE ABAS

Embora o uso de `sessionStorage` resolva a perda de dados ao trocar de aba, existe a necessidade de garantir que, se o usuário abrir a mesma solicitação em duas abas diferentes, não ocorra uma colisão de dados ou uma condição de corrida (race

condition). A sincronização de estado entre abas é um requisito avançado de UX que previne a corrupção de dados quando o operador trabalha com múltiplos contextos simultâneos.

A base legal para esta integridade reside no princípio da segurança da informação, onde a disponibilidade e a integridade devem ser mantidas em todos os pontos de acesso. O uso do *Broadcast Channel API* ou de eventos de `storage` do navegador deve ser considerado para notificar outras abas sobre mudanças críticas no formulário. Isso evita que um usuário finalize uma cotação em uma aba enquanto outra aba ainda mantém dados obsoletos em cache, o que poderia levar a erros de faturamento e logística.

A falha em sincronizar ou isolar corretamente os contextos de abas pode resultar em "dados fantasmas", onde informações de uma OS são salvas acidentalmente em outra solicitação. O desenvolvedor deve garantir que a chave do `sessionStorage` inclua um identificador único da sessão ou da OS (ex: `solicitacao-6906-dados`) para evitar sobreposição de informações entre diferentes atendimentos realizados simultaneamente.

15. IMPLEMENTAÇÃO DE LOGGING DETALHADO PARA DIAGNÓSTICO REMOTO

Para evitar que problemas de "OS não encontrada" ou "campos vazios" retornem sem uma explicação clara, é imperativo implementar um sistema de logging detalhado no lado do cliente e do servidor. Atualmente, o sistema falha sem deixar rastros (silent failure), o que impossibilita o diagnóstico rápido por parte da equipe de suporte. O *Marco Civil da Internet (Lei nº 12.965/2014)* estabelece a importância da manutenção de registros para fins de auditoria e segurança.

O racional técnico para o logging é capturar o estado exato da requisição: URL chamada, headers enviados (omitindo dados sensíveis), payload de resposta e tempo de latência. Estes logs devem ser enviados para um serviço de monitoramento (como Sentry ou LogRocket) ou armazenados em uma tabela de auditoria no banco de dados. Isso permitirá que, na próxima ocorrência de erro, o desenvolvedor saiba exatamente se a falha foi na API MubiSys, na rede, ou no processamento interno do sistema.

A ausência de logs técnicos será considerada uma obstrução ao processo de melhoria contínua. Cada falha de busca de OS deve gerar um log de nível `WARN` ou `ERROR` contendo o número da OS tentada e o motivo da falha (ex: "Token Expired", "OS Not Found in MubiSys", "Timeout"). Esta transparência de dados é o que diferencia um sistema amador de uma plataforma corporativa resiliente.

16. REESTRUTURAÇÃO DA BUSCA DE ORDEM DE SERVIÇO (ENDPOINT ESPECÍFICO)

A prática atual de buscar uma Ordem de Serviço através de um endpoint de listagem geral é tecnicamente ineficiente e logicamente incorreta. Conforme discutido na análise

de causa raiz, a listagem depende de filtros de data e paginação que podem ocultar a OS desejada. A arquitetura de APIs REST preconiza que recursos específicos devem ser acessados por identificadores únicos através de endpoints dedicados (Direct Access Pattern).

A fundamentação para esta mudança é a garantia de entrega do dado. Ao utilizar o endpoint `/ordem-servico/numero/{numero}`, eliminamos as variáveis de interferência como o range de 60 dias ou o status da OS. Esta é uma obrigação técnica de otimização de consulta: buscar pelo índice primário ou único é sempre preferível a filtrar uma lista em memória ou em banco. O descumprimento desta reestruturação manterá o sistema vulnerável a "regressões" aparentes que são, na verdade, apenas limitações do filtro de listagem.

As consequências de manter o fluxo atual incluem a insatisfação recorrente do usuário e a sobrecarga desnecessária da API externa. O desenvolvedor deve substituir a lógica de `find` em array de resultados por uma chamada direta ao recurso. Caso a API retorne 404, o sistema deve informar claramente: "OS não localizada na base da MubiSys", em vez de simplesmente não preencher os campos de CNPJ e Razão Social.

17. VALIDAÇÃO DE TIPOS E INTEGRIDADE COM TYPESCRIPT

A recorrência de erros de "undefined" sugere uma falha na tipagem dos dados que trafegam entre o backend e o frontend. O uso de `any` ou tipos mal definidos mascara erros que deveriam ser capturados em tempo de compilação. A robustez do sistema **Manus lago** depende de contratos de interface estritos (Interfaces/Types) que reflitam exatamente a estrutura de dados da API MubiSys e do banco de dados Prisma.

A base legal para a tipagem forte encontra-se na prevenção de erros sistêmicos e na segurança cibernética. Tipos mal definidos podem levar a vulnerabilidades de injeção ou corrupção de memória. O desenvolvedor deve definir esquemas de validação (usando bibliotecas como Zod ou Yup) para garantir que os dados recebidos da API possuam o formato esperado antes de serem injetados no estado do React. Se um CNPJ é esperado como string, o sistema deve validar essa premissa antes de tentar renderizá-lo.

A inobservância desta diretriz resultará em um código frágil e difícil de manter. A tipagem deve ser "end-to-end", garantindo que uma mudança na estrutura da OS no backend seja imediatamente sinalizada como um erro no frontend, impedindo o deploy de versões incompatíveis. Esta é a única forma de garantir que a funcionalidade de preenchimento automático de CNPJ e Razão Social não sofra retrocessos técnicos.

18. PLANO DE CONTINGÊNCIA PARA FALHAS DE INTEGRAÇÃO

Dado que o sistema depende de uma API de terceiros (MubiSys), é obrigatório o desenvolvimento de um modo de contingência ou "fallback". Se a API estiver fora do ar, o usuário não pode ser impedido de realizar a solicitação de frete manualmente. O sistema deve detectar a falha de integração e habilitar a edição manual dos campos de

CNPJ, Razão Social e Endereço, que normalmente seriam bloqueados para preenchimento automático.

Este requisito baseia-se no princípio da continuidade do negócio. Um bloqueio técnico em um fornecedor externo não pode paralisar a operação logística da empresa. O racional técnico envolve a implementação de um estado `isManualEntry` que é ativado quando a busca de OS falha ou quando o serviço externo está indisponível. Isso garante que a produtividade seja mantida mesmo em cenários de instabilidade tecnológica externa.

A ausência de um modo de contingência é uma falha de design de sistema. O desenvolvedor deve garantir que o formulário seja flexível o suficiente para aceitar dados manuais sob supervisão, registrando no banco de dados que aquela solicitação foi criada sem a validação automática da MubiSys. Esta rastreabilidade é essencial para auditorias futuras e para garantir que dados inseridos manualmente sejam verificados posteriormente.

19. CRONOGRAMA DE TESTES E HOMOLOGAÇÃO FINAL

A fase final de correção deve incluir um período de 48 horas de homologação em ambiente de *Staging* antes da promoção para *Production*. Durante este período, usuários-chave (Power Users) devem realizar o fluxo completo de solicitação de frete, testando especificamente os cenários de troca de aba e busca de OS em diferentes horários do dia. A aprovação formal desta etapa é condição *sine qua non* para o encerramento do ticket de correção.

A fundamentação para este rigoroso processo de teste é a eliminação definitiva das regressões. O histórico de "funciona e depois para de funcionar" indica que os testes anteriores foram superficiais ou realizados em condições ideais que não refletem o uso real. O cronograma deve prever testes de carga simulando múltiplos usuários acessando a página de solicitações simultaneamente para validar a eficácia do novo filtro de 30 dias.

Qualquer falha identificada durante a homologação reinicia o ciclo de correção. O compromisso da equipe de desenvolvimento é com a entrega de uma solução "zero defeitos" para os bloqueadores listados. A documentação dos testes, incluindo prints de tela e logs de sucesso, deve ser arquivada como evidência de cumprimento das instruções técnicas contidas neste relatório.

Documento elaborado em 08 de Agosto de 2026. As informações contidas são de responsabilidade do solicitante.

Local e data: São Paulo, 08 de Agosto de 2026